

Task-1

Subtask-1

Introduction: The Lucas–Kanade Algorithm & Implementation Overview

1. Theoretical Foundation: What is Lucas–Kanade?

Optical flow calculates the apparent motion of objects between consecutive video frames, caused by the relative movement between the camera and the scene. The Lucas–Kanade method is a highly popular differential algorithm for estimating *sparse optical flow*.

It relies on three core assumptions:

- **Brightness Constancy:**
A pixel from an object retains the same intensity in the next frame, even if it has moved.
 - **Small Motion:**
The displacement between consecutive frames is small enough to allow a Taylor series approximation (via image derivatives).
 - **Spatial Coherence:**
Neighboring pixels belonging to the same surface undergo similar motion.
-

Optical Flow Constraint Equation

From the brightness constancy assumption, we obtain:

$$I_x u + I_y v + I_t = 0$$

Where:

- I_x, I_y : Spatial image gradients (horizontal and vertical intensity changes)
 - I_t : Temporal gradient (change in intensity over time)
 - u, v : Unknown motion components in the x and y directions
-

Resolving the Ill-Posed Problem

Since there are two unknowns (u, v) but only one equation per pixel, the problem is ill-posed.

The Lucas–Kanade method resolves this using the spatial coherence assumption. By considering a local window (e.g., 25×25) around each pixel, we form an overdetermined system:

$$Av = b$$

Where:

- A : Matrix of spatial gradients (I_x, I_y)
 - $v = [u \ v]^T$: Motion vector
 - b : Vector of temporal gradients ($-I_t$)
-

Least Squares Solution

We solve using the least squares approach:

$$\begin{aligned} A^T A v &= A^T b \\ v &= (A^T A)^{-1} A^T b \end{aligned}$$

2. Code Implementation Breakdown

Instead of relying on OpenCV's built-in (black-box) functions, this implementation manually constructs and solves the Lucas–Kanade equations step by step:

Feature Initialization

We begin by detecting strong, trackable corners in the first frame using Shi–Tomasi corner detection (`cv.goodFeaturesToTrack`).

The parameters are tuned to prioritize high-contrast regions (e.g., clothing edges rather than soft facial features).

Raw Gradient Computation

For each new frame:

- Spatial gradients I_x and I_y are computed using a 3×3 Sobel operator.
- These gradients are scaled by dividing by 8.0 to maintain proper pixel-level consistency.

- The temporal gradient I_t is calculated as:

$$I_t = I_{\text{new}} - I_{\text{old}}$$

Constructing the Matrices

For each tracked feature point:

- A 25×25 window of gradients is extracted.
 - These 2D patches are flattened into 1D arrays.
 - The matrix A is formed from I_x and I_y , and vector b from I_t .
-

Solving and Filtering Outliers

- Compute $A^T A$.
- Before inversion, check its determinant:

$$|\det(A^T A)| < 10^{-6}$$

If true, the system is nearly singular (e.g., flat or textureless regions), so the point is discarded.

- Additionally, reject motion vectors where displacement exceeds $1.5 \times$ the window size, as these are physically implausible.
-

Dynamic Visual Tracking

Once valid motion vectors (u, v) are obtained:

- Update the feature point coordinates.
 - Maintain a rolling history of positions (`trail_length = 25`).
 - Draw continuous trajectories, creating a “*comet tail*” effect that visualizes motion paths without cluttering the frame.
-

Point Replenishment

As tracked points:

- Move out of frame, or
- Get rejected during filtering

...the system monitors the active track count.

If the number falls below 40, a background detection step is triggered to identify new features, ensuring consistent and dense tracking.

Subtask-1 Bonus

Introduction: The Horn–Schunck Algorithm & Implementation Overview

1. Theoretical Foundation: What is Horn–Schunck?

Optical flow estimates the apparent motion of objects between consecutive video frames due to relative motion between the camera and the scene.

The Horn–Schunck method is a classical *global (dense) optical flow algorithm*, meaning it computes motion vectors for every pixel in the image, unlike sparse methods such as Lucas–Kanade.

It is based on two key assumptions:

- **Brightness Constancy:**
The intensity of a pixel remains constant across consecutive frames.
- **Global Smoothness Constraint:**
The motion field varies smoothly across the entire image, meaning neighboring pixels have similar velocities.

Optical Flow Constraint Equation

From brightness constancy, we obtain:

$$I_x u + I_y v + I_t = 0$$

Where:

- I_x, I_y : Spatial image gradients
 - I_t : Temporal gradient
 - u, v : Horizontal and vertical motion components
-

Energy Minimization Framework

Horn–Schunck formulates optical flow as a global energy minimization problem:

$$E = \iint \left((I_x u + I_y v + I_t)^2 + \alpha^2 (|\nabla u|^2 + |\nabla v|^2) \right) dx dy$$

Where:

- First term enforces brightness constancy
 - Second term enforces smoothness of the flow field
 - α : Regularization parameter controlling smoothness
-

Iterative Solution

Instead of solving directly, Horn–Schunck uses an iterative approach:

- Initialize $u = 0, v = 0$
 - Repeatedly update flow using local neighborhood averages
 - Converge toward a smooth, globally consistent motion field
-

2. Code Implementation Breakdown

This implementation manually computes dense optical flow using the Horn–Schunck method, without relying on OpenCV's built-in functions.

Frame Preprocessing

- The video is loaded using `cv.VideoCapture`.
 - A specific segment of interest is selected (from 12s to 22s) to focus on a clean motion sequence.
 - Frames are resized into two resolutions:
 - Display Resolution: 800×450 (for visualization)
 - Computation Resolution: 200×112 (for faster processing)
 - Frames are converted to grayscale before processing.
-

Gradient Computation

For each frame pair:

- Spatial gradients I_x, I_y are computed using a 3×3 Sobel operator:

$$I_x = \frac{\partial I}{\partial x}, I_y = \frac{\partial I}{\partial y}$$

- These are scaled by dividing by 8.0 for proper normalization.
- Temporal gradient is computed as:

$$I_t = I_{\text{current}} - I_{\text{previous}}$$

- A light Gaussian blur is applied beforehand to reduce noise and improve derivative stability.
-

Initialization of Flow Fields

- Two matrices u and v (same size as the image) are initialized to zero.
 - These represent the horizontal and vertical motion for every pixel.
-

Local Averaging Kernel

A custom Laplacian-like kernel is defined:

$$\begin{bmatrix} \frac{1}{12} & \frac{1}{6} & \frac{1}{12} \\ \frac{1}{6} & 0 & \frac{1}{6} \\ \frac{1}{12} & \frac{1}{6} & \frac{1}{12} \end{bmatrix}$$

This kernel computes the local average of neighboring flow vectors, enforcing smoothness.

Iterative Flow Update

For a fixed number of iterations (e.g., 40–50):

1. Compute local averages:

$$u_{\text{avg}}, v_{\text{avg}}$$

2. Compute update term:

$$P = \frac{I_x u_{\text{avg}} + I_y v_{\text{avg}} + I_t}{\alpha^2 + I_x^2 + I_y^2}$$

3. Update flow:

$$u = u_{\text{avg}} - I_x P$$
$$v = v_{\text{avg}} - I_y P$$

- The denominator is precomputed for efficiency.
 - The parameter α controls the trade-off between data fidelity and smoothness.
-

Scaling Flow to Display Resolution

- The computed flow (low resolution) is resized back to display size.
 - Motion vectors are scaled appropriately using a scale factor to maintain correct magnitude.
-

Visualization Using HSV Color Space

To visualize dense flow:

- Convert (u, v) into:
 - Magnitude (speed)
 - Angle (direction)
 - Map these to HSV color space:
 - Hue $\rightarrow$ Direction
 - Value $\rightarrow$ Speed
 - Saturation $\rightarrow$ Maximum (255)
 - Convert HSV to BGR for display.
-

Overlay and Display

- The flow visualization is blended with the original frame:

$$\text{Output} = 0.5 \cdot \text{Frame} + 0.8 \cdot \text{Flow}$$

- The result is displayed in real-time.
 - Playback speed depends on computational cost (this method is CPU-intensive).
-

Loop Control and Termination

- Processing continues until:
 - End of selected video segment is reached, or
 - User presses 'q'
 - Frames are updated iteratively for continuous flow estimation.
-

Key Characteristics of This Implementation

- Fully manual implementation of Horn–Schunck (no built-in optical flow functions)
- Produces dense motion fields (motion at every pixel)
- Uses iterative global optimization
- Trades accuracy for speed via resolution downscaling
- Computationally heavier than sparse methods but provides richer motion information

Subtask-2

1. Theoretical Foundation: System Overview

This system implements an autonomous driving agent in a simulated environment using:

- Sparse Optical Flow (Lucas–Kanade) → for motion perception
- Color-based Segmentation → for scene understanding
- Artificial Potential Fields (APF) → for decision making

The primary objectives are:

- Follow a target (blue wall/path)
 - Avoid obstacles using optical flow
 - Stay within safe road boundaries (avoid grass)
-

Key Concept: Artificial Potential Fields (APF)

The navigation strategy is based on combining multiple virtual forces:

- Attractive Force: Pulls the agent toward a goal
- Repulsive Force: Pushes the agent away from obstacles
- Boundary Constraints: Keeps the agent within safe limits

The final steering command is:

$$\theta = \theta_{\text{attractive}} + \theta_{\text{repulsive}} + \theta_{\text{boundary}}$$

2. Code Implementation Breakdown

Simulation Setup

- Environment initialized using PyBullet (setup_simulation)
 - Car model includes:
 - Steering joints
 - Motor joints
 - Runs in real time with controlled stepping
-

Camera & Perception Pipeline

- Camera mounted slightly ahead and above the car
- Oriented along forward direction
- Frames captured using p.getCameraImage

Processing steps:

- Resize to 320 × 240
- Convert to grayscale (optical flow)

- Convert to HSV (color detection)
-

Sparse Optical Flow (Custom Lucas–Kanade)

- Gradients computed using Sobel:

$$I_x, I_y$$

- Temporal gradient:

$$I_t = I_{\text{current}} - I_{\text{previous}}$$

- For each feature point:

- Extract 21×21 window
- Construct A, b
- Solve:

$$v = (A^T A)^{-1} A^T b$$

- Accept only if:

$$\det(A^T A) > 10^{-4}$$

Feature Detection

- Uses Shi–Tomasi Corner Detection
 - Restricted to yellow regions (road) via HSV mask
-

Visual Preprocessing: Horse Blinders

- Left/right margins masked
 - Ignored during flow computation
 - Improves focus on central region
-

3. Control Strategy (Artificial Potential Fields)

1. Attractive Force (Target Following)

$$x_{\text{target}} = \frac{M_{10}}{M_{00}}$$
$$\theta_{\text{attractive}} = (x_{\text{center}} - x_{\text{target}}) \cdot k$$

Memory Mechanism

$$x_{\text{memory}} = 0.95 \cdot x_{\text{memory}} + 0.05 \cdot x_{\text{center}}$$

Prevents erratic motion when the target is lost.

2. Boundary Repulsion (Grass Avoidance)

$$\theta_{\text{boundary}} \propto \text{distance to boundary}$$

Keeps the car centered and stable.

3. Repulsive Force (Obstacle Avoidance)

$$w = e^{k \cdot \text{magnitude}} - 1$$
$$\theta_{\text{repulsive}} = (F_{\text{right}} - F_{\text{left}}) \cdot k$$

4. Subsumption Architecture

If:

$$|\theta_{\text{repulsive}}| > 0.1$$

Then:

$$\theta_{\text{attractive}} \rightarrow 0.1 \cdot \theta_{\text{attractive}}$$

5. Steering Smoothing

$$\theta_{\text{final}} = 0.6 \cdot \theta_{\text{current}} + 0.4 \cdot \theta_{\text{target}}$$

4. Visualization & Debugging

- Optical flow vectors (green)
- Target (blue)
- Memory target (gray)
- Boundaries (red)
- Horse blinders
- Yellow mask preview

Metrics shown:

- Total steering
 - Boundary correction
 - Repulsive force
-

5. Control Loop Execution

Each iteration:

1. Step simulation
 2. Capture frame
 3. Compute optical flow
 4. Detect features/targets
 5. Compute forces
 6. Combine & smooth steering
 7. Apply control
-

6. Technical Implementation Report

1. Motion Estimation & Gradient Scaling

Problem: "Single-Level Motion Blur"

Without a pyramidal Lucas–Kanade approach, the system struggled with large pixel displacements at high speeds.

Solution:

- Implemented frame skipping (3 physics steps per vision step)
- Applied 5×5 Gaussian blur

This effectively spreads gradients, allowing tracking of motion beyond the 1–2 pixel limitation.

2. Feature Selection & Environmental Noise

Problem: "Schizophrenic Sky" Error

The tracker locked onto horizon lines and floor textures, falsely detecting global motion.

Solution:

- Switched to HSV color segmentation
- Applied a yellow mask

This made the system blind to irrelevant regions, isolating true obstacles.

3. Control Hierarchy & Vector Conflict

Problem: "Tug-of-War"

Attractive and repulsive forces canceled each other, causing weak avoidance.

Solution:

- Implemented Subsumption Architecture
 - Repulsion suppresses attraction by 90% during threats
-

4. Feedback Loops & Ego-Motion

Problem: "Death Spiral"

Camera rotation from steering caused false perception of increasing obstacle motion.

Solution:

- Introduced Horse Blinders (Ignore Margins)
- Ignored edge regions (40–70 pixels)

This broke the feedback loop and stabilized behavior.

5. Artificial Potential Field (APF) Equilibrium

Problem: "Grass is Lava"

Car overcorrected and drifted off track.

Solution:

- Created Equilibrium Potential Well
- Added virtual repulsive walls using:

$$F = K \cdot d$$

This keeps the vehicle centered on the road.

7. Final System Hierarchy

The agent operates using a three-layer behavioral hierarchy:

1. Safety (Hard Constraint)

- Repulsion from road boundaries
- Prevents leaving the track

2. Avoidance (Reactive)

- Exponential repulsion from obstacles
- Based on optical flow

3. Navigation (Proactive)

- Attraction toward blue target
 - Uses memory-based smoothing
-

Key Characteristics

- Integrates computer vision + control systems
- Uses manual Lucas–Kanade optical flow
- Achieves real-time autonomous navigation

Robustness

- Handles target loss
- Works under noise

- Adapts to dynamic environments

Demonstrates

- Full Perception → Decision → Control pipeline
- Practical robotics application of optical flow